

Claude Code Cheatsheet — `/goal` & `orchestrating-swarms`

Two ways to make Claude Code work on its own across many turns. One-pager for Atram engineers — last updated 2026-05-22.

`/goal` **built-in · v2.1.139+**

WHAT IT DOES

Sets a completion condition. Claude keeps working turn after turn without prompting; an evaluator model (Haiku by default) checks the transcript after each turn and decides whether to stop. Clears automatically when the condition holds.

COMMANDS

```
/goal <condition> # set & start
/goal # status: turns, tokens,
evaluator reasoning
/goal clear # stop early
```

SESSION BEHAVIOR

- Persists across turns within the session
- Survives `--resume` if the session ends mid-goal
- Does **not** survive new sessions (unlike hooks)
- Works in headless mode (`-p` flag)

WRITING GOOD CONDITIONS

- Phrase as something **visible in the transcript** — the evaluator can't run shell commands or read files
- Always include a turn cap: `... or stop after 20 turns`
- Make it boolean: `passes/fails`, `exits 0/non-zero`, `clean/dirty`

Gotcha: *"the deploy succeeded"* won't work unless Claude actually prints the deploy output. Frame goals around things Claude will show.

`orchestrating-swarms` **skill · community**

WHAT IT IS

A marketplace skill (compound-engineering family) that wraps Claude Code's native `Task` / `Agent` tool and `TeammateTool` into a coordinated multi-agent workflow. Not a built-in command.

WHAT IT DOES

- Spawns multiple subagents in parallel
- Dependency-aware task ordering (B waits on A)
- File-backed JSON state so swarms survive across turns/sessions
- Inbox-style messages between agents
- A "leader" approves plans and coordinates shutdown

HOW TO INVOKE

```
/compound-engineering:orchestrating-swarms
```

The skill emits the swarm protocol; the parent agent fans out work via the `Task` tool and the swarm tracks state in a local JSON store.

WHEN TO REACH FOR IT

- The work is genuinely parallel (3+ independent chunks)
- You want reproducible runs you can re-enter later
- You need cross-agent messaging, not just fan-out

Don't use it for: a single linear task, or anything `/goal` + one agent can handle. Swarms add coordination overhead.

Where they pay off at Atram

`/goal` Flaky test hunt

`/goal pytest tests/alerts runs 10 times in a row with all green, or stop after 25 turns` — Claude re-runs, inspects failures, patches, re-runs. You read the diff at the end.

`/goal` Lint-clean a branch

`/goal ruff check . and eslint . both exit 0, or stop after 20 turns` — useful before opening a PR. Bound the turn count or it'll grind on whitespace forever.

swarms Multi-repo audit

One agent per repo (yaya-engine, yaya-frontend, atram-utils-webapp, alert-manager) — each scans for hardcoded secrets, reports back to a leader that aggregates into one issue.

swarms Weather model evaluation sweep

N agents each run a different forecast variant against the same eval set, write results to shared JSON state. Leader builds a comparison table.

`/goal` Migration backfill verification

`/goal supabase-py count on users_v2 equals count on users for every country, or stop after 15 turns` — runs the comparison, identifies drift, prints reasoning, halts on parity.

`/goal` Headless CI helper

In `-p` mode on a CI runner: `/goal all yaya-engine integration tests pass, or stop after 12 turns`. Pipes diagnostics back without you babysitting.

swarms Parallel code review

Three reviewers (security, performance, Rails-style) work the same diff at once; leader consolidates findings, deduplicates, posts a single PR comment.

swarms WhatsApp template translation

One agent per language (EN, ES, AM, SW) translates the same template set in parallel, leader validates placeholders match across all outputs before committing.